

Active Reproducibility for File-Based Exploratory Science

Anthony Westbrook
University of New Hampshire
Durham, New Hampshire, USA
anthony.westbrook@unh.edu

Lakshmi Ongolu
University of New Hampshire
Durham, New Hampshire, USA
Lakshmi.Ongolu@unh.edu

Bocheng Cui
University of New Hampshire
Durham, New Hampshire, USA
Bocheng.Cui@unh.edu

Elizabeth Varki
University of New Hampshire
Durham, New Hampshire, USA
varki@unh.edu

Abstract

Computational reproducibility is commonly understood as regenerating a file's exact contents by re-executing the workflow that produced it. In exploratory science, however, workflows are rarely explicit: computation evolves through backtracking, parameter tuning, and abandoned execution paths, so the operations that produce a final result are often not represented in any single, coherent workflow description. As a result, irreproducibility is not merely a failure of documentation or user discipline, but a systems problem of recovering the correct workflow from unconstrained execution.

We introduce Active Reproducibility, a model that generalizes this goal beyond byte-identical output, defining reproducibility through three complementary guarantees: exact replay in the original environment, semantic equivalence across environments, and computational fidelity under changing inputs. We realize this model in RepeatFS, a file system that records unconstrained computational activity and reconstructs a minimal causal workflow for each output file. It then replays this workflow across machines by automatically resolving duplicate processes and translating hardcoded local paths, without containers or manual setup.

In a multi-day bioinformatics experiment, RepeatFS isolates 18 relevant processes from 579 exploratory tasks while bypassing more than 114,000 concurrent system processes. Across a benchmark of 18 published experiments, it successfully replicates 13 of 14 viable workflows from their recorded lineage, including on machines other than the one where they were recorded. Its overhead is concentrated in I/O-intensive workloads, while compute-bound tasks experience minimal slowdown.

CCS Concepts

• **Information systems** → **Data provenance**; • **Software and its engineering** → *File systems management*; *Software verification and validation*.

Keywords

Computational Reproducibility, Replicability, Active Reproducibility, Data Provenance, Causal Lineage, User-Space File Systems, Workflow Reconstruction, Exploratory Science

ACM Reference Format:

Anthony Westbrook, Bocheng Cui, Lakshmi Ongolu, and Elizabeth Varki. 2026. Active Reproducibility for File-Based Exploratory Science. In *ACM Conference on Reproducibility and Replicability (ACM REP '26)*, July 20–22, 2026, Delft, Netherlands. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3820002.3828580>

1 Introduction

Reproducibility in computational research is the ability to regenerate a file's exact contents by re-executing the workflow that produced it. Because this is central to scientific validity, numerous tools have been developed to support it. These broadly fall into two categories: *archival tools* and *workflow management systems*. Archival tools such as ReproZip [14], Docker, and Olive [35] capture executions along with their environments (binaries, libraries, and data) into immutable containers or virtual-machine images. Workflow systems such as Snakemake [27], Nextflow [19], and VisTrails [10] instead require users to explicitly encode pipelines as dependency graphs or scripts.

Despite these advances, reproducibility remains elusive. In a survey of over 1,500 researchers, more than 70% reported failing to reproduce others' experiments, and more than half failed to reproduce their own [6]. In practice, re-executing a recorded workflow often produces different results.

This gap arises because existing tools target finalized workflows, while much of scientific work occurs during *phase-1 exploratory* analysis. Computational research is inherently iterative and non-linear: scientists develop pipelines through ad-hoc trial and error, executing commands interactively, tuning parameters, and revising intermediate results. By the time a final output is produced, its *provenance* - the precise combination of inputs, parameters, code versions, and environment state - is often obscured by this exploratory process.

Existing tools struggle in this setting because they assume that users can provide a complete and correct workflow specification. In practice, this assumption fails for two reasons. First, human memory is fallible: users may omit hidden dependencies or mistakenly capture failed trials instead of successful ones. Second, workflows are influenced by external factors beyond user control, such as silent updates to shared system libraries or databases. As a result,



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

ACM REP '26, Delft, Netherlands

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2778-8/26/07

<https://doi.org/10.1145/3820002.3828580>

even faithfully captured workflows may be incorrect. This is the garbage in, garbage out problem of reproducibility: a containerized error remains an error.

To address this limitation, we introduce *Active Reproducibility*, a formal model that shifts the goal of reproducibility from preserving executions to reconstructing correct workflows. The model defines reproducibility through three complementary guarantees: (i) exact replay in local environments, (ii) semantic equivalence across heterogeneous systems, and (iii) computational fidelity under changing inputs. Achieving these guarantees requires reconstructing workflows at the granularity of per-file causal lineage, rather than relying on user-specified pipelines or coarse-grained tracing.

We realize this model in RepeatFS, a user-space system that automatically reconstructs per-file causal lineage from unconstrained exploratory executions. RepeatFS continuously records command-line activity and file operations, and extracts precise, causally inferred per-file lineages on demand from raw provenance logs. It further enables automated cross-machine replication, visualization of workflow structure, and validation of execution correctness by identifying points of divergence.

This work focuses on file-based exploratory workflows, where data is processed through sequences of non-interactive command-line tools. Unlike archival systems that capture a finalized pipeline as a static artifact, RepeatFS treats reproducibility as an ongoing process, maintaining provenance across sessions and time. This design reflects how research evolves in practice, often across collaborators and heterogeneous computing environments.

The primary contribution of this paper is the definition and formalization of Active Reproducibility for exploratory, file-based workflows, along with its realization in a practical system. Specifically, we make the following contributions:

- **Active Reproducibility Model:** A formal model of reproducibility based on three guarantees - exact replay, cross-environment semantic equivalence, and computational fidelity - and the insight that these require reconstruction at the level of per-file causal lineage.
- **Automatic Recording and On-Demand Extraction:** A log-raw, build-later approach that records unconstrained exploratory activity and dynamically extracts precise per-file causal lineages from noisy provenance logs only when requested.
- **Portable Replication:** The ability to re-execute extracted per-file lineages across heterogeneous environments without containers or manual environment reconstruction.
- **Verification and Introspection:** Integrated visualization and validation mechanisms that expose divergence points, enabling users to analyze and refine evolving workflows.

2 Exploratory Scenario

We present a motivational example from the field of Bioinformatics to illustrate the reality of exploratory computational research and the specific challenges that must be addressed by RepeatFS. This scenario highlights three critical obstacles: the non-linear messiness of human exploration, the opacity of script-based execution (including hidden dependencies), and the difficulty of extracting causal lineages.

Example: A researcher intends to identify and annotate genetic variants from a patient's DNA sample. The workflow involves a mix of automated scripts and interactive exploration, spanning multiple days with unrelated tasks.

```
# 1. BATCH PROCESSING: Run standard alignment
  pipeline
# The user executes a script 'align_prep.sh':
#   bwa index ref.fa
#   bwa mem ref.fa reads.fq > aligned.sam
#   samtools view -b aligned.sam > data.bam
$ sh align_prep.sh

# 2. ATTEMPT 1 (Mistake): Call variants with defaults
$ samtools sort -o data.sorted.bam data.bam
$ bcftools mpileup -f ref.fa -Ob -o data.bcf data.
  sorted.bam
$ bcftools call -mv data.bcf > variants.vcf
$ grep -c "INDEL" variants.vcf
50000 # User realizes this is too noisy

# 3. FILTERING (Futile): filter the noisy file
$ bcftools filter -i 'TYPE="snp"' variants.vcf >
  final_snps.vcf

# 4. INSPECTION: Check the result
$ wc -l final_snps.vcf
# Result is still too large. User abandons this path.
$ rm final_snps.vcf variants.vcf data.bcf ref.ra.fai

# --- [Time Gap: 3 Days elapse] ---
# User logs in multiple times for other work.
# The shell history buffer may be flushed.
# -----

# 5. ATTEMPT 2 (Correction): Call with strict filters
# User decided to add more filters
$ bcftools mpileup -f ref.fa -Ob -a DP,AD,ADF,ADR,SP,
  QS -g 1 -d 10000 -Q 0 -q 0 data.sorted.bam > data.
  .bcf
$ bcftools call -mv data.bcf > variants.vcf

# 6. FILTERING: Extract final candidates
# Note: Overwrites the previous 'final_snps.vcf'
$ bcftools filter -i 'TYPE="snp"' variants.vcf >
  final_snps.vcf

# 7. ANNOTATION: Predict gene effects
# Challenge: Implicitly reads global config file
$ snpEff ann -noStats human_g1k_v37 final_snps.vcf >
  annotated.vcf
```

This explorative research scenario highlights the following challenges:

- (1) **Messiness and Discontinuity:** The researcher types commands related to their research over a period of days, interleaved with unrelated commands. The researcher also

re-executes commands multiple times. For instance, the file `variants.vcf` is first written in Step 2 and then overwritten in Step 5. The provenance log must record all these actions chronologically.

(2) Script Opacity and Hidden Dependencies:

- *Script Loss*: A naive script recorder would only capture the command `sh align_prep.sh` in Step 1. If the researcher modifies or loses the `align_prep.sh` file after execution, the provenance is broken. RepeatFS must capture the child processes (e.g., `bwa`, `samtools`), allowing the workflow to be rebuilt even if the original wrapper script is lost.
- *Implicit Inputs*: A naive script recorder would miss that `bwa mem` relies on five auxiliary index files (`.bwt`, `.pac`, etc.) output by `bwa index` the first command in `align_prep.sh`. These files are never explicitly named, yet they are required for execution. RepeatFS must capture information about all files that are read and written by processes via system calls. If a researcher deletes one of these files, it will result in failure.
- *Implicit Configuration*: Step 7 relies on a global configuration file for `snpeff` to define the genome database locations. If the researcher accidentally updates the config file or the collaborator's machine has a config that points to a different genome build, the command will execute successfully but produce invalid results. RepeatFS must record the read operation on this external configuration file to flag divergences.

(3) Distinguishing Provenance from Lineage: Finally, the system must separate the signal from the noise.

- *Temporal Provenance* records the entire session, including the failed parameter choice in Step 2 and the manual inspection via `grep` and `wc`.
- *Causal Lineage* constructs the clean workflow by tracing backward from the writer of `annotated.vcf`. RepeatFS identifies dependencies on Step 7, Step 6, Step 5, and the internal steps of Step 1. Crucially, it prunes Step 2 (the error), the unrelated work between Steps 4 and 5, and Step 3 (the abandoned path).

3 Active Reproducibility

In the context of exploratory science, where code, data, and environments are in constant flux, definitions of reproducibility that require identical binary outputs are impractical. Instead, we define active reproducibility in terms of scientific content. A result is reproducible if its scientific content remains invariant under *platform variation* (i.e., changes in the machine, library versions, or file metadata) and changes only in response to *material variation*, namely changes to the input data or to the workflow that produced the result (its tools, parameters, and sequence of operations).

Consequently, this definition requires the underlying workflow (tools, parameters, and sequence) to be *deterministic*. Non deterministic computations, such as Monte Carlo simulations without fixed seeds or race-prone concurrent programs, naturally fall outside this guarantee, as their scientific content can vary even when the data and workflow are held fixed. For such computations, RepeatFS guarantees the faithful re-execution of the commands (with the

correct inputs and parameters), but it cannot guarantee invariant content.

We argue that a result is scientifically reproducible if the system can guarantee this invariance across three distinct levels of execution context:

1. *Exact Identity (Same Machine, Same Context)*. When the lineage is re-executed on the original machine with the original inputs and environment, RepeatFS guarantees bitwise identity. Since the inputs, libraries, paths, and environment variables are unchanged, the resulting files must yield the identical MD5 hash. Any deviation here signals an error in the provenance capture or local filesystem corruption.

2. *Semantic Equivalence (Different Machine, Same Input)*. When the lineage is transferred to a collaborator's machine (or a different cluster node), the environment shifts. System libraries (e.g., `libmath`) may differ in version, and file headers may capture new timestamps or paths. Consequently, the MD5 hashes of the processes and results in the reconstructed lineage will differ from the values recorded in the original provenance log. However, RepeatFS guarantees semantic equivalence: because the input data and the workflow logic (the tool parameters and sequence) are identical, the scientific content of the output remains invariant. For example, a variant caller running on a different computer may produce a binary-different VCF file (e.g., due to differing metadata headers recording the new execution environment), but it must detect the exact same set of genetic variants.

3. *Computational Fidelity (Changing Inputs)*. A key feature of active reproducibility is the decoupling of logic from data. In exploratory research, a reproduction often involves applying the same method to updated data (e.g., re-running the analysis on a newer reference genome, like replacing `ref.fa v1` with `v2`). In this scenario, the output is expected to change entirely. However, RepeatFS guarantees computational fidelity: the exact sequence of operations, parameter choices, and filtering thresholds used in the original lineage are applied automatically to the new input. This ensures that any change in the result is attributable to the data update, not to an accidental drift in the workflow logic.

Based on these three execution contexts, we propose the following definition:

A file is reproducible if the workflow that produced it can be reconstructed from its causal lineage and re-executed, yielding a result that satisfies Exact Identity in the original context, Semantic Equivalence across different environments, and Computational Fidelity under changing inputs.

To translate this conceptual definition into practice, we require a computational framework. The following section presents the RepeatFS reproducibility model, which establishes these architectural specifications.

4 Reproducibility Model

Prior work often assumes that capturing provenance - a passive record of computational history - is sufficient for reproducibility [3, 31, 34]. We argue that while provenance provides the necessary data, it lacks the execution logic required to reconstruct the workflow

that produced a result in a new environment. To bridge the gap between “knowing what happened” and “making it happen again,” RepeatFS introduces a Reproducibility Model that pairs provenance data capture with active re-execution strategies.

Designing this model for file reproducibility requires bridging the gap between the researcher’s high-level intent and the low-level reality of system execution. While capturing high-level commands is essential for automated re-execution, these processes are often black boxes with hidden dependencies. As demonstrated in our motivational example (Section 2), a command like `bwa mem` relies on auxiliary index files (such as `.bwt` and `.pac`) that are never explicitly named in the script, yet are required for execution. To capture these hidden dependencies, a system must observe low-level system calls; however, blindly tracing all OS activity incurs prohibitive overhead and background noise.

RepeatFS addresses this cost–benefit tension by recording high-level commands together with user-space file system operations. Our model posits that this combined information is sufficient to reconstruct complete causal lineages and thereby reproduce files. However, because raw logs alone are insufficient to handle the challenges of messy, exploratory science, RepeatFS must augment them with specific structural logic:

- (1) Exploratory Noise (Causal Inference): the model must use causal inference to isolate a target file’s lineage from unrelated background activity and to distinguish successive instances of the same file produced by overwrites (Sections 4.1 and 4.2).
- (2) Process-Tree Redundancy (Double Execution): because a captured trace records both a parent process and its `fork/execve` children, the model must resolve this redundancy - replaying the parent or synthesizing the workflow from child commands - to avoid executing the same work twice (Section 4.3).
- (3) Environmental Path Dependencies: since native re-execution cannot assume a fixed filesystem layout, the model must detect hardcoded paths in the lineage and virtualize them to the target environment (Section 4.4).

We next describe the design principles that enable our model to transform raw provenance into executable reproducibility.

4.1 Per-File Lineage

For file-based exploratory workflows, in which a process’s output is determined by its input files, tracking file system operations provides sufficient granularity to guarantee reproducibility while treating processes as black boxes. For every system call that accesses a file, RepeatFS records metadata about the process (e.g., arguments, environment) and the file, broadly classifying these calls into two categories:

- Read operations: Calls that read data or metadata (e.g., `read`, `stat`, `open`).
- Write operations: Calls that modify a file’s data (e.g., `write`, `truncate`).

From the recorded events, RepeatFS constructs a lineage graph where nodes represent Files and Processes, and edges represent causal dependencies (Reads and Writes).

To reconstruct the lineage of a specific file, RepeatFS identifies the processes that write to the file and recursively traces their input

files. However, a simple structural graph is insufficient; causality implies a temporal ordering. RepeatFS records the timestamp of every system call to enforce the happens-before constraint: a file instance depends on inputs read prior to the write that produced that instance. For example, consider a process that reads file *X* at t_1 , writes to file *Y* at t_2 , and subsequently reads file *Z* at t_3 . A naive recorder might list both *X* and *Z* as inputs for *Y* simply because they were accessed by the same process. By utilizing timestamps, RepeatFS correctly identifies that *Y* depends on *X* ($t_1 < t_2$) but prunes *Z* ($t_3 > t_2$).

4.2 File Instances

Files are frequently overwritten during exploratory research. As illustrated in the motivational example, the file `variants.vcf` is created in Step 2, found to be erroneous, and subsequently overwritten in Step 5.

RepeatFS does not preserve physical copies (snapshots) of every file version, as this would incur prohibitive storage overhead. Instead, to prevent the loss of lineage associated with earlier versions, RepeatFS represents each overwrite as a permanent, distinct node in the provenance graph. Whenever a process opens a file in a mode that creates or truncates it (e.g., `O_CREAT` or `O_TRUNC`), RepeatFS records a new, distinct File Instance node in the provenance log.

Consequently, when Step 5 overwrites `variants.vcf`, the log stores this as a new instance, while the record of the previous instance (from Step 2) remains intact. This ensures that a researcher can reconstruct the older, abandoned instance not by retrieving a backup, but by re-executing the lineage steps preserved in the provenance log.

4.3 Process-Tree Redundancy

A major challenge in system-level provenance capture is handling hierarchical process trees, which occur whenever a program spawns child processes via standard system calls like `fork` and `execv`. While a common exploratory example is a researcher executing a wrapper script (e.g., `sh align_prep.sh`) that subsequently launches several computational tools (e.g., `bwa`, `samtools`), this architectural challenge applies to any parent process. A naive system-call recorder logs both the parent and all of its children as independent events. During replication, this leads to double execution: the system may re-execute the high-level parent command while also explicitly replaying its recorded child commands.

To resolve this process-tree redundancy, RepeatFS records the complete process hierarchy and identifies a *Session Leader* for each interactive command. The session leader is the top-level command initiated by the user. RepeatFS then tracks all descendant processes spawned from it, whether through scripts, pipes, subshells, or internal application logic. By maintaining these parent–child relationships, RepeatFS intelligently determines the correct replay strategy: it executes the top-level session leader (which naturally re-spawns the necessary children) and safely prunes the redundant child commands from the replay queue. Alternatively, if the original parent environment or script is lost, RepeatFS can synthesize the workflow by explicitly executing the granular sequence of child commands to faithfully reconstruct the underlying scientific logic.

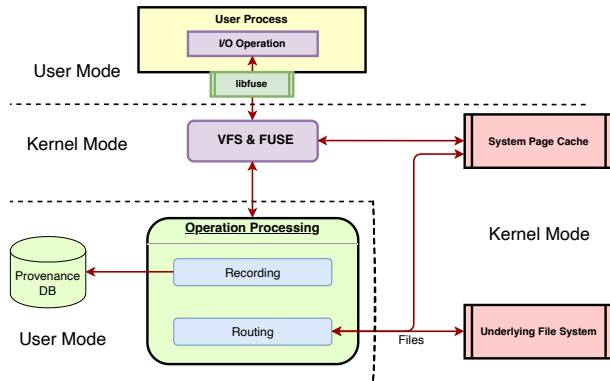


Figure 1: RepeatFS's Internal Flow

4.4 Dynamic Path Virtualization

A major barrier to reproducibility in native (non-containerized) environments is the hardcoded path problem. An analysis recorded at `/home/userA/project` often fails when re-executed at `/home/userB/project` because absolute paths are inevitably embedded in scripts, command-line arguments, and environment variables.

RepeatFS's design principle for reproducibility decouples the computational logic from the file system layout. RepeatFS introduces a lightweight virtualization layer to achieve portability without environment isolation. During replication, the system intercepts the execution context of every process, specifically parsing the command-line arguments, environment variables, and current working directory (CWD). It dynamically identifies substrings matching the original recording environment (e.g., `/home/userA/`) and transparently substitutes them with the valid path of the current environment (e.g., `/home/userB/`). This mirrors the link rewriting performed by web archives such as the Internet Archive's Wayback Machine, which rewrite the absolute URLs embedded in an archived page so they resolve within the archived snapshot rather than escaping to the live web: in both settings, absolute references captured in one context must be transparently rewritten to resolve correctly in another.

This approach avoids the data sequestration common in containerized or archival solutions. In tools like ReproZip, outputs are generated within an isolated directory structure (chroot) or virtual image, requiring an export step to be usable. By contrast, RepeatFS's dynamic path rewriting directs output immediately to the user's native workspace. This supports a Replicate-and-Extend workflow: a collaborator can reproduce a predecessor's file by re-executing its lineage, and immediately continue the analysis using their own local tools, without extracting data from a frozen bundle.

5 Implementation

RepeatFS [40] is implemented as a user-space file system (FUSE) that overlays transparently onto an existing directory tree [38]. Figure 1 presents the internal flow of RepeatFS. The complete Python source code, installation instructions, and setup scripts are available on the project's GitHub repository.¹

¹<https://github.com/ToniWestbrook/repeatfs>

RepeatFS employs a Passthrough architecture where a virtual mount point mirrors an existing physical directory. For example, a user mounts RepeatFS at `~/mount`, designating `~/project` as the backing store. Consequently, the entire project hierarchy becomes accessible within the mount point, where RepeatFS automatically intercepts metadata operations (e.g., `open`, `close`) but passes the actual data directly to the underlying physical file system. The user must operate within this mount point to enable tracking. This architecture allows researchers to activate provenance tracking on datasets instantly, without migrating files, wrapping execution commands, or installing custom kernels.

This section details the log-raw, build-later strategy that distinguishes RepeatFS from prior causality-based provenance systems like PASS [31]. Additionally, we describe how this architecture realizes the active reproducibility model proposed in Section 4.

5.1 Provenance Schema and Capture

A primary challenge in system-level provenance is balancing capture granularity with performance overhead. Recording every individual `read()` or `write()` system call generates unmanageable logs and prohibitive latency. Conversely, constructing a dependency graph in kernel memory before logging incurs high memory overhead and complexity. RepeatFS addresses this tension by decoupling logging from graph construction.

Instead of first constructing a lineage graph in memory, RepeatFS logs raw metadata immediately into a lightweight relational database (SQLite). The schema, illustrated in Figure 2, consists of four normalized tables: `Process`, `File`, `Read`, and `Write`.

5.1.1 Capturing Process Context. Although RepeatFS intercepts activity at the file system layer, every operation FUSE delivers is associated with the process that issued it: FUSE supplies the calling process's identifier (along with its user and group IDs) with each request. Using this identifier, RepeatFS reads the corresponding process metadata exposed by the kernel through the `/proc` interface: the full command line and arguments, the executable path, the environment, the current working directory, and the parent, thread-group, and session identifiers. RepeatFS records them in the `Process` table (Figure 2). Command-line arguments are therefore obtained directly from the executing process rather than reconstructed from the shell.

5.1.2 Optimized I/O Logging. Synchronously recording every individual `read` or `write` in the database would introduce high latency. To eliminate this runtime overhead, RepeatFS consolidates continuous I/O activity into a single database transaction committed only upon file closure.

For each unique pair of (`Process`, `File`), the system tracks read/write metadata in memory, maintaining a time range (`start`, `stop`) and a bitmask of operations (`ops`). For example, if a process issues 1,000 writes to a file over 5 seconds, RepeatFS simply updates the `stop` timestamp in memory during the burst. The actual I/O write to the provenance database occurs only once, when the file is closed. This strategy decouples the application's execution speed from the logging mechanism, significantly reducing performance overhead while simultaneously compressing log size. This open-to-close batching mirrors the session semantics of whole-file-caching

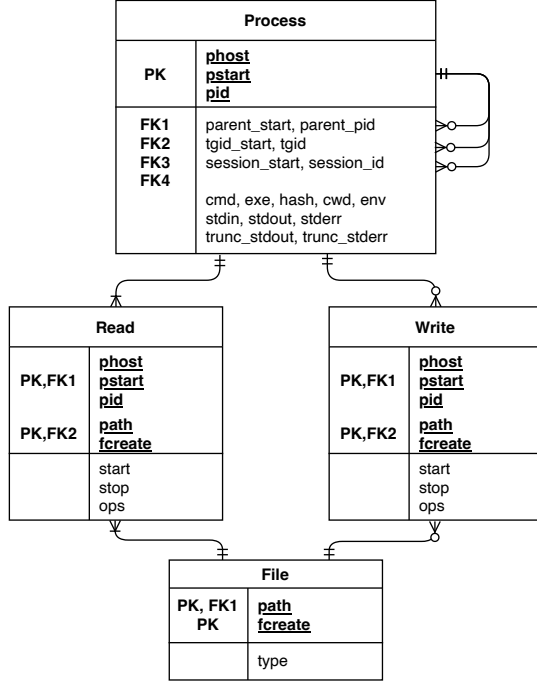


Figure 2: Provenance Schema (PK: Primary Key; FK: Foreign Key)

distributed file systems such as AFS [25], in which byte-level reads and writes are serviced directly against a locally cached copy and the client contacts the server only when a file is opened or closed.

5.1.3 Handling File Instances. To support the File Instance model of Section 4, the File table uses a composite primary key of the file path and a creation timestamp (*fcreate*). Whenever RepeatFS intercepts an `open()` with `O_CREAT` or `O_TRUNC`, it inserts a new row, so a file created early in a workflow and a later version that overwrites it are recorded as distinct instances. A researcher can therefore request the provenance of, and regenerate, any specific prior instance, even one later modified or deleted.

Using these flags is a deliberate heuristic: `O_CREAT` and `O_TRUNC` mark the points at which a file’s contents are wholly replaced, so the new instance’s lineage is correctly disconnected from any prior content. A process that instead modifies bytes in place, without truncating, only partially changes the file; its new state still depends on the prior content, so RepeatFS correctly does not reset the lineage; it records the modification as a write against the existing instance, preserving both that dependency and the happens-before ordering. The heuristic is thus conservative: it cuts lineage only on unambiguous total replacement and never drops a genuine dependency. Its one cost is that successive in-place modifications are not exposed as separately regenerable generations, which does not

affect re-execution, since the full chain of writers to the instance is retained.

5.2 On-Demand Per-File Lineage

RepeatFS logs raw provenance and constructs the lineage of a file only on-demand. The causal graph is constructed only when a user requests the provenance of a specific file. This shifts the computational cost from the active execution path to the passive phase of analysis.

When a query is issued for a target file (e.g., `results.txt`), RepeatFS executes a recursive traversal algorithm that applies structural logic to the raw logs:

- (1) Target identification: The system queries the File table for the entry matching the file path and the latest timestamp, isolating the correct instance.
- (2) Causal filtering: It retrieves all processes from the Write table that modified this file instance. Notably, it filters these writes using the Happens-Before constraint, ensuring that only writes completing before the file’s current state are included.
- (3) Recursive expansion: For each identified writer process, the system queries the Read table to find its input files, recursively repeating the process until the graph is complete. Recursion along a path stops when a process has no recorded input reads or when a file has no recorded writer.

5.3 Automated Export and Replay

Once the lineage is reconstructed, RepeatFS exports it on demand as two portable files per target (e.g., `data.txt.json` and `data.txt.html`). The JSON file holds the complete machine-readable execution graph (including high-level scripts and all low-level child processes), and drives replication. The HTML file embeds this data in a self-contained interactive SVG for validation, letting researchers inspect the workflow topology in any browser without special software. Because these two files are generated on demand from the provenance store rather than cached as static copies, each is always consistent with the current causal lineage of its target file [40]. A complete example of this JSON lineage, and its rendered HTML visualization, is available in the project GitHub repository.

5.3.1 Replay Engine Mechanisms. During the replication phase, the RepeatFS replay engine parses the exported JSON lineage to enforce the structural guarantees defined in Section 4. To resolve process-tree redundancy, the engine dynamically applies Session Leader logic at runtime: it defaults to executing only the top-level session leaders (pruning redundant child records), or provides a script-recovery mode that explicitly synthesizes missing parent scripts from the recorded child sequences. Concurrently, the client performs user-space path virtualization by scanning the JSON execution context for original path prefixes (e.g., `/home/userA/`) and dynamically substituting them with the absolute path of the active mount point (e.g., `/home/userB/`). This ensures seamless cross-machine portability directly in the native workspace, without requiring chroot or containerized isolation.

5.3.2 Integrity Validation. Beyond simple re-execution, RepeatFS ensures computational identity via a cryptographic verifier. Crucially, RepeatFS validates the *processes* rather than the files. During provenance capture, the system records the MD5 hash of every executable binary and the execution topology, but intentionally omits hashing of input/output files. (MD5 here detects accidental change, not adversarial tampering; it is pluggable, and SHA-256 can be substituted where adversarial integrity is required.) This design supports active reproducibility (Section 3): it allows researchers to verify that a workflow was re-executed with identical logic (same binaries, same parameters) even when applied to new or changing input datasets.

After replaying the workflow, the system performs validation to ensure fidelity:

- (1) Executable integrity: Using a PID map generated during replay, RepeatFS aligns the new processes with their original counterparts. It compares the MD5 hash of each executable binary against the original record. This allows the system to flag silent failures caused by environment shifts (e.g., a hidden library update or a mismatched tool version) while ignoring differences in input file content.
- (2) Topology verification: RepeatFS recursively validates the execution structure. It orders the original child processes by timestamp and verifies that the replicated process spawned the same number of children in the exact same order. If a branch deviates (e.g., an extra child process or a missing step), the system emits a warning for that branch while continuing to verify the rest of the graph.

This mechanism transforms reproducibility from a simple “it ran/it failed” outcome into a diagnostic tool. By verifying that the logic (executables) and structure (topology) match the original while permitting data variations, RepeatFS validates that the workflow execution was faithful, regardless of the inputs used.

5.3.3 Interactive Provenance Visualization. Provenance graph images are often illegible for real-world workflows containing thousands of nodes. RepeatFS instead exports lineage as a self-contained interactive HTML/SVG document that runs in any standard web browser, without the original GUI required by centralized workflow systems. A collapsible tree layout lets researchers move between a global view of the high-level data flow and drilled-down subgraphs of child processes and intermediate files (e.g., expanding a `.bam` node to reveal the `samtools sort` commands that produced it), while hovering over any node shows its captured execution context, namely, command-line arguments, environment variables, and timestamps. This lets collaborators audit a pipeline’s logic without installing the original software stack.

6 Experiments

Our evaluation assesses RepeatFS across three dimensions: (1) its ability to isolate relevant causal lineage from noisy exploratory environments, (2) its robustness in replicating real-world experiments from published literature, and (3) its performance overhead on standard file-processing workloads.

Unless otherwise specified, all experiments were performed on a shared high-performance computing (HPC) server to accurately

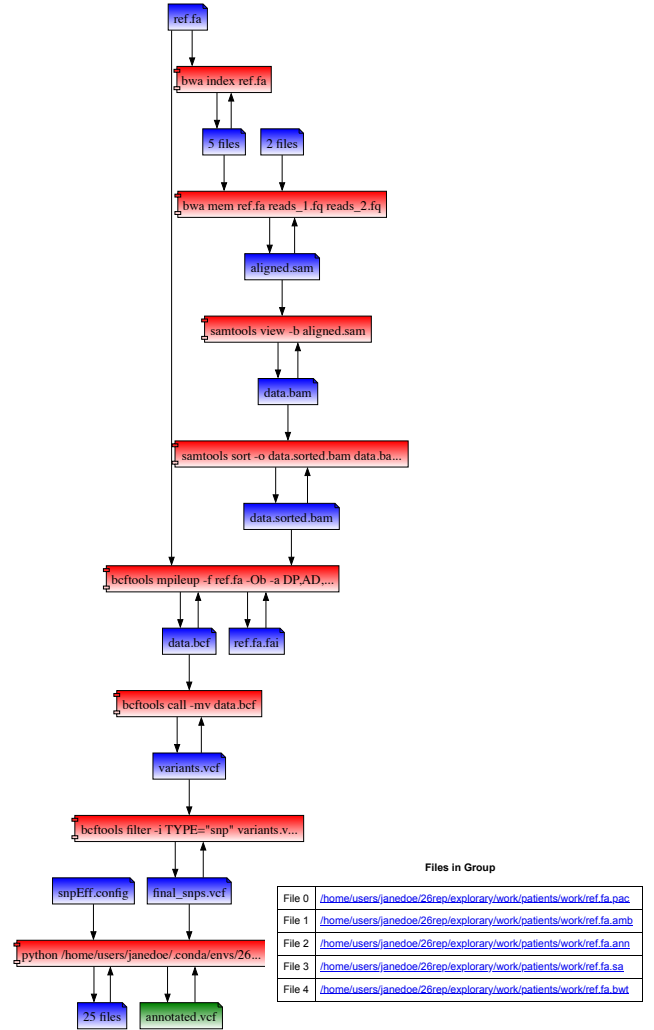


Figure 3: Lineage extracted by RepeatFS for annotated .vcf in the motivational example of Section 2. The interactive graph, automatically generated and rendered by RepeatFS, isolates the critical processes (red) and file dependencies (blue) that produced the target output (green) from the surrounding multi-day exploratory noise. The interactive box displays the contents of a condensed node (e.g., the 5-file node), previewing them on hover and listing them on click.

reflect the noisy, multi-user conditions of real-world scientific computing. The server is shared among multiple users, and all tracking was conducted with limited user-level permissions. It is equipped with an AMD EPYC 9754 processor (256 cores, 2.25 GHz base frequency) and 3.0 TiB of RAM. The system runs Rocky Linux 9.5 (Blue Onyx) with kernel version 5.14.0. The storage configuration consists of a 49 TB XFS HDD used to store all input, output, and provenance data, a 102 TB ZFS volume, and a smaller NVMe SSD for the operating system and boot.

6.1 Extracting Lineage from Exploratory Noise

To evaluate RepeatFS's ability to separate experimental signal from background noise, we revisit the bioinformatics scenario of Section 2: a 48-hour, highly non-linear session in which the researcher abandons an initial variant-calling attempt, interleaves unrelated tasks (protein alignment with Paladin [41], preprocessing with Trimmomatic [8], and assembly with MEGAHIT [29]), and only later re-runs the corrected analysis.

Rather than untangle this history by hand, the researcher queries RepeatFS for the causal lineage of the final annotated.vcf. RepeatFS extracts the minimal correct workflow in three forms: a JSON execution trace for automated replication, a synthesized shell script for independent execution, and an interactive provenance graph (Figure 3) that can be zoomed from the high-level process chain down to individual file dependencies.

Table 1 quantifies the filtering. The monitored directory accumulated 5,846 files and 579 processes over the two days; RepeatFS pruned this to the 193 processes causally related to the researcher's work, and to just 18 processes (2.1 GB) for the core variant-calling pipeline. Monitoring system-wide execution over the same period logged 114,194 concurrent processes. Isolating 18 critical processes from 579 user tasks while bypassing more than 114,000 system-level events illustrates the cost RepeatFS avoids: by scoping capture to the user-space mount point and building lineage on demand, it isolates the scientific signal without the overhead of whole-system logging.

6.2 Reproducing Published Literature

To evaluate RepeatFS's cross-machine capabilities under real-world conditions, we sampled 18 diverse experiments from the CompRep dataset [16, 17]. Table 2 summarizes the evaluation metrics, where *Experiment Size* denotes the total disk storage footprint of the downloaded repository, including input datasets, scripts, and output files.

First, we established a functional baseline by attempting to manually execute all 18 experiments. Four workflows (E7, E12, E15, and E17) failed this initial baseline due to missing critical input files or degraded external resources. Notably, this baseline attrition included all experiments relying on R, driven primarily by the fragmentation and decay of historical R package repositories [7, 11]. Despite substantial effort to reconstruct the specified `renv` environments, legacy dependencies could not be resolved. This reduced our viable testing pool to 14 functional experiments.

For the 14 viable experiments, our evaluation followed a two-phase protocol: first, we executed the original scripts under RepeatFS to transparently record their execution provenance; second,

we used RepeatFS's automated replay engine to replicate the entire workflow from the extracted JSON lineage. Under this protocol, RepeatFS successfully captured and replicated 13 of the 14 workflows. The single automated replication failure was E14. While E14 could be executed manually, its reliance on interactive user prompts at runtime rendered it non-deterministic, making it ineligible for RepeatFS's automated replay engine.

During the automated replication phase, we also encountered an environmental conflict: E13 and E16 required Python 2.7, which directly conflicted with RepeatFS's Python 3.0 dependency under the restricted user-level permissions of our shared HPC environment. To validate RepeatFS's dynamic path virtualization and cross-environment portability, we exported the JSON lineages of these two experiments to the isolated workstation (described in Section 6.3), where both workflows replicated from the JSON lineage file.

While RepeatFS excels at virtualizing paths and resolving process tree redundancies across diverse architectures, it intentionally does not archive underlying system binaries or package ecosystems. The baseline failure of the legacy R experiments underscores a broader lesson for exploratory science: guaranteeing long-term replication requires coupling structural provenance tools like RepeatFS with stable, version-pinned package management environments or executable-archival approaches such as Olive [35], which preserve the binaries and environments RepeatFS intentionally does not capture.

6.3 Performance Evaluation

To assess RepeatFS's overhead during realistic file-processing workloads, we evaluated five standard tasks: (1) file copying (`cp`); (2) compression/decompression (`zip/unzip` with a ~ 0.7 ratio); (3) AES-256 encryption/decryption; (4) photo editing (processing RAW files in `darktable-cli`, resizing with `magick`, and JPEG encoding via `cjpeg`); and (5) video transcoding (H.264 to H.265 via `ffmpeg`).

We measured each task under four configurations, shown as the four bars per task in Figure 4. These configurations isolate three distinct sources of overhead: the FUSE framework (Native $\rightarrow$ FUSEpy), provenance recording (FUSEpy $\rightarrow$ RepeatFS), and lineage replay (Replicate).

To avoid HPC background noise, we benchmarked on an isolated workstation (Intel Core i7-10700, 16 GB RAM, Fedora Linux 40). To simulate standard storage conditions, the OS ran on an NVMe SSD while all experimental I/O and provenance data were routed to a 7200 RPM HDD. As Figure 4 illustrates, write-intensive tasks (e.g., Zip compression) incur the highest overhead, whereas compute-bound workloads exhibit minimal degradation. The small gap between the FUSEpy and RepeatFS bars shows that provenance recording adds little beyond the framework itself; most of RepeatFS's absolute slowdown versus native Linux instead stems from FUSEpy's lack of a write-back cache, demonstrating RepeatFS's overall efficiency.

Our benchmarks exercise throughput- and write-intensive workloads but not the metadata-intensive regime, where user-space file systems are known to incur their highest relative overhead [39] and which RepeatFS's open/close interception would most directly

Table 1: Exploratory Reproducibility

Experiment	Files	Processes	I/O	Size
BWA (motivation)	78	18	120	2.1G
Encryption	25	22	54	65G
Decryption	35	27	79	46G
Paladin	404	98	1258	55G
Genome assembly	114	28	205	80M
Total (experiments)	656	193	1716	168.18G
Total (watching directory)	5846	579	24028	174G

Table 2: Replication of Published Experiments.

Baseline indicates whether the experiment could be executed manually from its original repository.

Replicated indicates whether RepeatFS successfully re-executed it from its recorded lineage.

The **Repo link** column provides a clickable hyperlink directing to the original source code and dataset repository (e.g., GitHub, Zenodo, Bitbucket) for each experiment.

Experiment ID	Publication	Repo link	Programming Language	Env	Files	Experiment Size	Baseline	Replicated
E1	Adnan et al. [1]	Link	Unix shell, Python	Conda	57	4.6 GB	Yes	Yes
E2	Bai and Zhao [5]	Link	Python	Conda	50	241.5 MB	Yes	Yes
E3	Chauhan et al. [12]	Link	C++	System	71	125.6 MB	Yes	Yes
E4	Sun et al. [37]	Link	Python /PostgreSQL	Conda	3636	364.9 MB	Yes	Yes
E5	Zhou et al. [44]	Link	C++	System	11	86.2 MB	Yes	Yes
E6	Chen et al. [13]	Link	Python	System	85	2.4 MB	Yes	Yes
E7	Kukucka et al. [28]	Link	Java, Python, Unix Shell	-	791	153.4 MB	No ¹	-
E8	Xie et al. [42]	Link	Python	Conda	66	10.4 MB	Yes	Yes
E9	Jehn [26]	Link	Python	Conda	64	2.7 MB	Yes	Yes
E10	Lin et al. [30]	Link	Python	Conda	208	128.8 MB	Yes	Yes
E11	Agresta et al. [2]	Link	Python	Conda	6	352.8 KB	Yes	Yes
E12	Collins et al. [15]	Link	R	-	101	147.9 MB	No ¹	-
E13	Hai [24]	Link	Python /SQLite	Conda	26	68.6 KB	Yes	Yes
E14	Dritsa et al. [20]	Link	Jupyter Notebook	Conda	59988	1.1 GB	Partial ²	No*
E15	Cai and Smeeding [9]	Link	R	-	31	4.6 MB	No ¹	-
E16	Spinellis et al. [36]	Link	R /Python /Unix Shell	Conda	15	327.9 KB	Yes	Yes
E17	Almås et al. [4]	Link	R	-	4062	183.7 MB	No ¹	-
E18	Fronchetti et al. [21]	Link	Python	Conda	6409	134.4 MB	Yes	Yes

¹ Repository fragmentation or missing files. ² Insufficient detail.

* Experiment is non-deterministic.

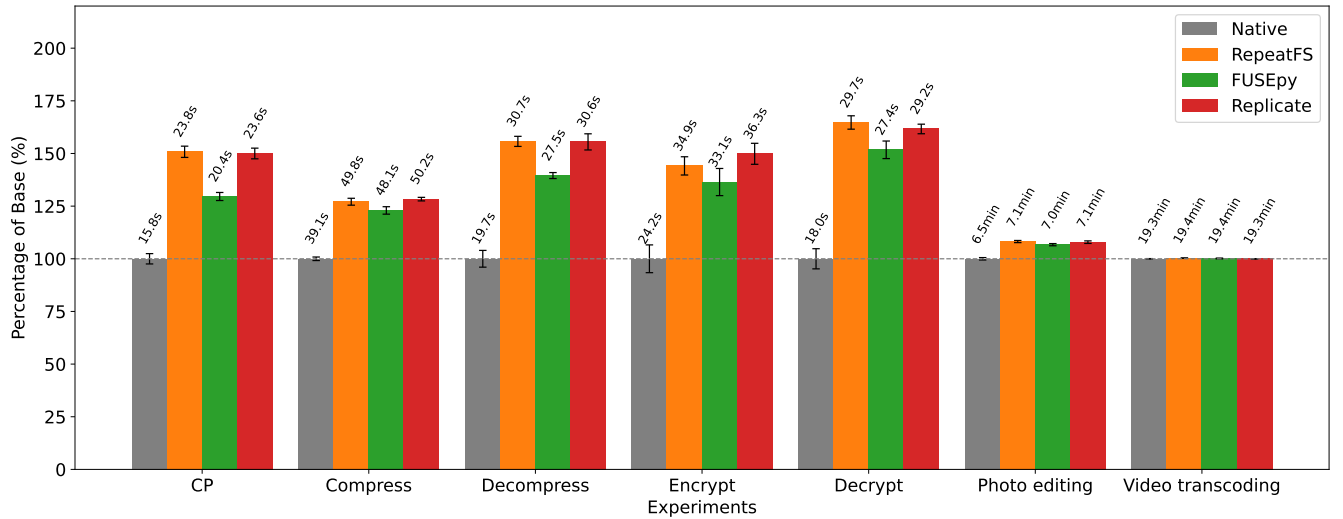


Figure 4: Performance across five file-processing workloads, reported as runtime relative to the native baseline. Each task is measured under four configurations: *Native* (no FUSE layer), *FUSEpy* (bare FUSE passthrough, provenance disabled), *RepeatFS* (provenance recording enabled), and *Replicate* (the task re-executed from its extracted JSON lineage via RepeatFS’s replicate command).

stress; a metadata-heavy evaluation, reported with trial counts and variance, is left to future work.

6.4 Summary

Lineage isolation, a prerequisite for reproducibility, is demonstrated by distilling 18 critical processes from 579 exploratory tasks.

Exact Identity (Level 1), corresponding to same-machine replay, is demonstrated by re-executing 13 of 14 workflows on the recording host and verifying executable and topology integrity.

Semantic Equivalence (Level 2), corresponding to cross-machine replay, is demonstrated by experiments E13 and E16. Their Python 2.7 dependencies could not run on the shared HPC environment, so they were exported from their JSON lineage and successfully replicated on a different workstation.

Computational Fidelity (Level 3), re-applying a fixed workflow to changed inputs, is supported by design rather than directly evaluated. RepeatFS validates process binaries and execution topology, but not input or output files, allowing workflows to be verified as logically faithful while their data vary. A direct evaluation of Level 3 is left to future work, as it requires varying inputs while comparing against a known-correct outcome.

7 Related Work

RepeatFS builds upon prior research in computational reproducibility, provenance capture, and workflow management. We categorize prior systems into three primary approaches: archival tools for portability, workflow management systems for structured execution, and system-level provenance for auditing.

7.1 Archival and Portability Tools

Tools like ReproZip [14], Olive [35], and CDE [33] excel at retrospective packaging to ensure portability across different environments. Once a researcher has finalized a workflow, ReproZip traces the execution to identify all file and library dependencies, bundling them into a self-contained package (e.g., a Docker container or .rpz file). This wrapper approach is highly effective for portability, ensuring a finished result can run on another machine. However, these tools are designed for the end of the research lifecycle. They require finalized workflows, making them less suitable for the exploratory phase where workflows are emerging and untested. In contrast, RepeatFS operates continuously in the background, capturing the lineage of intermediate, failed, and successful attempts without requiring the user to define a workflow upfront.

7.2 Workflow Management Systems

Systems such as VisTrails [10], Snakemake [27], and Taverna [43] prioritize semantic structure. They require users to explicitly define their analysis as a graph of dependencies, often using a specialized GUI or domain-specific language (DSL). This approach offers excellent clarity and capabilities: users can visualize the pipeline, modify parameters, and re-execute specific stages. However, they impose a structure-first burden: researchers must formalize their logic before they can execute it. In exploratory science, where the correct workflow is unknown until the analysis is complete, this requirement can be a significant barrier. RepeatFS inverts this model by adopting a structure-later approach: we allow users to work naturally in the

shell, and we automatically reconstruct the structural graph from the observed execution trace.

7.3 System-Level Provenance and Auditing

Kernel-level and passthrough systems like PASS [31], SPADE [22], and CamFlow [32] focus on fidelity and auditing. By intercepting operations at the VFS or kernel level, they capture a comprehensive graph of all system activity. PASS, for example, pioneered the concept of automatic, transparent capture by constructing lineage graphs in kernel memory in real-time. While this eager approach ensures nothing is missed, it incurs the complexity of managing graph structures within the kernel.

In contrast, RepeatFS logs the raw metadata immediately and defers the computationally expensive graph construction until the user actually requests it. This allows RepeatFS to remain lightweight and portable while matching the automatic capture benefits of kernel-level predecessors. Burrito [23] shares RepeatFS's record-everything, reconstruct-later philosophy, capturing system-wide activity for later inspection. Unlike RepeatFS, it is not file-system-based and targets lab-notebook-style documentation and annotation rather than automated, portable re-execution.

While prior systems provide the fundamental data required for reproducibility, their primary design goal is often auditing, security, or compliance rather than active user interaction. Consequently, they rarely expose tools for researchers to easily re-execute a specific file's lineage or interactively visualize the derivation of a result. RepeatFS adapts the high-fidelity capture mechanisms of these systems but pairs them with a user-centric query engine designed specifically for on-demand replication and visualization.

7.4 Hybrid Reproducibility Systems

More recent provenance systems, such as Eidetic [18], ReAnimator [3], and Ursprung [34], have moved beyond provenance capture to re-execution and analysis.

Eidetic records non-deterministic system inputs to enable time-travel debugging, allowing users to retrospective query the memory and register state of past processes. ReAnimator captures storage-level traces with high fidelity to replay exact I/O patterns. This approach is ideal for benchmarking or debugging complex race conditions, where the goal is to recreate the precise internal state, rather than scientific workflow reproduction.

Ursprung addresses the semantic gap in provenance by combining system-level provenance with application-level logs to support data science pipelines. This hybrid approach allows researchers to query the lineage across system and application boundaries. However, Ursprung is primarily designed for auditing and analysis: helping users understand what happened.

These systems often rely on complex infrastructure or application specific instrumentation. RepeatFS differentiates itself by operating entirely in user-space and focusing on portable, application-agnostic re-execution.

The RepeatFS Approach: RepeatFS occupies a unique design point by synthesizing the automatic capture of PASS, the portability goals of ReproZip, and the interactive visualization of VisTrails.

However, the unconstrained nature of exploratory science necessitated a fundamental architectural redesign regarding how these goals are achieved.

8 Conclusion

We defined the Active Reproducibility model and implemented it in RepeatFS, a system that automatically extracts the minimal, correct workflow from unconstrained exploratory executions using causal lineage. Our evaluation demonstrates that RepeatFS can isolate relevant computation from substantial exploratory noise, distilling 18 critical processes from 579 tasks while avoiding the overhead of tracking over 114,000 concurrent system processes. Across a benchmark of 18 published experiments, RepeatFS replicated 13 of 14 viable workflows from their recorded lineage, including on machines other than the one where they were recorded, demonstrating practical, cross-machine reproducibility in real-world settings.

RepeatFS validates the fidelity of a workflow's logic (its binaries and execution topology), rather than the correctness of its output, so a faithfully replayed workflow can still yield an incorrect result when a tool is non-deterministic or its inputs are flawed. The model targets file-based, non-interactive command-line workflows and does not address interactive, GUI, or network-mediated computation. RepeatFS also does not archive binaries or package ecosystems, so durable reproducibility requires pairing it with version-pinned environments or archival.

RepeatFS's cross-machine support is further confined to a single operating system. Because operating systems differ substantially in their system-call interfaces, filesystem semantics, and dependency management, our implementation specifically targets Linux. Cross-OS reproducibility is more practically achieved through containers or virtual machines (e.g., using Olive or running RepeatFS inside a VM). While the Active Reproducibility model is not itself OS-specific, building a cross-platform implementation would require a substantial engineering effort for little practical gain.

RepeatFS's lineage extraction is intentionally deterministic and rule-based: the guarantees of Active Reproducibility require that re-execution depend only on the input data and the workflow, so the reproducibility-critical path must not introduce nondeterministic inference. AI-assisted methods are nonetheless promising for peripheral tasks that leave these guarantees intact, such as summarizing or labeling extracted workflows, suggesting human-readable names for intermediate files, or explaining the points of divergence flagged during validation. We leave such uses to future work.

Finally, RepeatFS currently relies on fine-grained temporal ordering within a local provenance store to construct causal lineage. As future work, we plan to extend this approach to distributed environments by replacing physical time with logical time and causal ordering, enabling accurate lineage reconstruction across decentralized systems.

Acknowledgments

We thank the anonymous reviewers for their careful reading and comments; incorporating their suggestions greatly improved this paper.

References

- [1] Muhammad Adnan, Yassaman Ebrahimzadeh Maboud, Divya Mahajan, and Prashant J. Nair. 2021. Accelerating recommendation system training by leveraging popular choices. *Proc. VLDB Endow.* 15, 1 (Sept. 2021), 127–140. doi:10.14778/3485450.3485462
- [2] Antonio Agresta, Chiara Biscarini, Fabio Caraffini, and Valentino Santucci. 2023. An Intelligent Optimised Estimation of the Hydraulic Jump Roller Length. In *Applications of Evolutionary Computation*, João Correia, Stephen Smith, and Raneem Qaddoura (Eds.). Springer Nature Switzerland, Cham, 475–490.
- [3] Ibrahim Umit Akgun, Geoff Kuenning, and Erez Zadok. 2020. Re-Animator: Versatile High-Fidelity Storage-System Tracing and Replaying. In *Proceedings of the 13th ACM International Systems and Storage Conference* (Haifa, Israel) (SYSTOR '20). Association for Computing Machinery, New York, NY, USA, 61–74. doi:10.1145/3383669.3398276
- [4] Ingvald Almås, Alexander W. Cappelen, Erik Ø. Sørensen, and Bertil Tungodden. 2022. Global evidence on the selfish rich inequality hypothesis. *Proceedings of the National Academy of Sciences* 119, 3 (2022), e2109690119. arXiv:https://www.pnas.org/doi/pdf/10.1073/pnas.2109690119 doi:10.1073/pnas.2109690119
- [5] Jiyang Bai and Peixiang Zhao. 2021. TaGSim: type-aware graph similarity learning and computation. *Proc. VLDB Endow.* 15, 2 (Oct. 2021), 335–347. doi:10.14778/3489496.3489513
- [6] Monya Baker. 2016. 1,500 scientists lift the lid on reproducibility. *Nature* 533, 7604 (May 2016), 452–454. doi:10.1038/533452a
- [7] Gabriel Becker, Coty Barr, Robert Gentleman, Michael Lawrence, and Dirk Edlbuettel. 2017. Enhancing Reproducibility and Collaboration via Management of R Package Cohorts. *Journal of Statistical Software* 82, 1 (2017), 1–20. doi:10.18637/jss.v082.i01
- [8] Anthony M. Bolger, Marc Lohse, and Bjoern Usadel. 2014. Trimmomatic: a flexible trimmer for Illumina sequence data. *Bioinformatics* 30, 15 (04 2014), 2114–2120. arXiv:https://academic.oup.com/bioinformatics/article-pdf/30/15/2114/4892471/bioinformatics_30_15_2114.pdf doi:10.1093/bioinformatics/btu170
- [9] Yixia Cai and Timothy Smeeding. 2020. Deep and Extreme Child Poverty in Rich and Poor Nations: Lessons from Atkinson for the Fight Against Child Poverty. *Italian Economic Journal* 6, 1 (March 2020), 109–128. doi:10.1007/s40797-019-00116-w
- [10] Steven P. Callahan, Juliana Freire, Emanuele Santos, Carlos E. Scheidegger, Cláudio T. Silva, and Huy T. Vo. 2006. VisTrails: visualization meets data management. In *Proceedings of the 2006 ACM SIGMOD International Conference on Management of Data* (Chicago, IL, USA) (SIGMOD '06). Association for Computing Machinery, New York, NY, USA, 745–747. doi:10.1145/1142473.1142574
- [11] Chung-hong Chan and David Schoch. 2023. rang: Reconstructing reproducible R computational environments. *Plos one* 18, 6 (2023), e0286761.
- [12] Komal Chauhan, Kartik Jain, Sayan Ranu, Srikanta Bedathur, and Amitabha Bagchi. 2021. Answering regular path queries through exemplars. *Proc. VLDB Endow.* 15, 2 (Oct. 2021), 299–311. doi:10.14778/3489496.3489510
- [13] Zhuangbin Chen, Jinyang Liu, Yuxin Su, Hongyu Zhang, Xiao Ling, Yongqiang Yang, and Michael R. Lyu. 2022. Adaptive performance anomaly detection for online service systems via pattern sketching. In *Proceedings of the 44th International Conference on Software Engineering* (Pittsburgh, Pennsylvania) (ICSE '22). Association for Computing Machinery, New York, NY, USA, 61–72. doi:10.1145/3510003.3510085
- [14] Fernando Chirigati, Rémi Rampin, Dennis Shasha, and Juliana Freire. 2016. ReproZip: Computational Reproducibility With Ease. In *Proceedings of the 2016 International Conference on Management of Data* (San Francisco, California, USA) (SIGMOD '16). Association for Computing Machinery, New York, NY, USA, 2085–2088. doi:10.1145/2882903.2899401
- [15] Stephen Collins, Gavin Smart, Ben Albright, and David Crippin. 2016. Food Inspection Evaluation Predictions - Source Code. <https://github.com/Chicago/food-inspections-evaluation>. [Online; accessed February-2024].
- [16] Lázaro Costa, Susana Barbosa, and Jácome Cunha. 2025. CompRep: A Dataset For Computational Reproducibility. In *Proceedings of the 3rd ACM Conference on Reproducibility and Replicability* (Vancouver, Canada) (ACM REP '25). Association for Computing Machinery, New York, NY, USA, 168–178. doi:10.1145/3736731.3746160
- [17] Lázaro Costa, Susana Barbosa, and Jácome Cunha. 2025. Curated dataset of 18 computational experiments (E1-E18). doi:10.5281/zenodo.15492423
- [18] David Devecsery, Michael Chow, Xianzheng Dou, Jason Flinn, and Peter M. Chen. 2014. Eidetic Systems. In *11th USENIX Symposium on Operating Systems Design and Implementation* (OSDI 14). USENIX Association, Broomfield, CO, 525–540. <https://www.usenix.org/conference/osdi14/technical-sessions/presentation/devcsery>
- [19] Paolo Di Tommaso, Maria Chatzou, Evan W. Floden, Pablo Prieto Barja, Emilio Palumbo, and Cedric Notredame. 2017. Nextflow enables reproducible computational workflows. *Nature Biotechnology* 35, 4 (2017), 316–319. doi:10.1038/nbt.3820

- [20] Konstantina Dritsa, Thodoris Sotiropoulos, Haris Skarpetis, and Panos Louridas. 2020. Search Engine Similarity Analysis: A Combined Content and Rankings Approach. In *Web Information Systems Engineering – WISE 2020*, Zhisheng Huang, Wouter Beek, Hua Wang, Rui Zhou, and Yanchun Zhang (Eds.). Springer International Publishing, Cham, 21–37.
- [21] Felipe Fronchetti, David C. Shepherd, Igor Wiese, Christoph Treude, Marco Aurélio Gerosa, and Igor Steinmacher. 2023. Do CONTRIBUTING Files Provide Information about OSS Newcomers' Onboarding Barriers?. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (San Francisco, CA, USA) (ESEC/FSE 2023). Association for Computing Machinery, New York, NY, USA, 16–28. doi:10.1145/3611643.3616288
- [22] Ashish Gehani and Dawood Tariq. 2012. SPADE: support for provenance auditing in distributed environments. In *Proceedings of the 13th International Middleware Conference* (Montreal, Quebec, Canada) (Middleware '12). Springer-Verlag, Berlin, Heidelberg, 101–120. doi:10.1007/978-3-642-35170-9_6
- [23] Philip J. Guo and Margo Seltzer. 2012. BURRITO: wrapping your lab notebook in computational infrastructure. In *Proceedings of the 4th USENIX Conference on Theory and Practice of Provenance* (Boston, MA) (TaPP'12). USENIX Association, USA, 7.
- [24] Ton Hai. 2019. Incremental Query Execution. <https://bitbucket.org/TonHai/iqe/src/master/> [Online; accessed February 2024].
- [25] John H. Howard, Michael L. Kazar, Sherri G. Menees, David A. Nichols, M. Satyanarayanan, Robert N. Sidebotham, and Michael J. West. 1988. Scale and performance in a distributed file system. *ACM Trans. Comput. Syst.* 6, 1 (Feb. 1988), 51–81. doi:10.1145/35037.35059
- [26] Florian Ulrich Jehn. 2021. *zutm/Extreme-Climate-Change: Publication ready*. doi:10.5281/zenodo.4588383
- [27] Johannes Köster and Sven Rahmann. 2012. Snakemake—a scalable bioinformatics workflow engine. *Bioinformatics* 28, 19 (Aug. 2012), 2520–2522. doi:10.1093/bioinformatics/bts480
- [28] James Kukucka, Luis Pina, Paul Ammann, and Jonathan Bell. 2022. CONFETTI: amplifying concolic guidance for fuzzers. In *Proceedings of the 44th International Conference on Software Engineering* (Pittsburgh, Pennsylvania) (ICSE '22). Association for Computing Machinery, New York, NY, USA, 438–450. doi:10.1145/3510003.3510628
- [29] Dinghua Li, Chi-Man Liu, Ruibang Luo, Kunihiko Sadakane, and Tak-Wah Lam. 2015. MEGAHIT: an ultra-fast single-node solution for large and complex metagenomics assembly via succinct de Bruijn graph. *Bioinformatics* 31, 10 (01 2015), 1674–1676. arXiv:https://academic.oup.com/bioinformatics/article-pdf/31/10/1674/49013263/bioinformatics_31_10_1674.pdf doi:10.1093/bioinformatics/btv033
- [30] Shuqi Lin, Donald Pierson, Robert Ladwig, Benjamin Kraemer, and Fenjuan Hu. 2023. Multi-Model Machine Learning Approach Accurately Predicts Lake Dissolved Oxygen with Meteorological and Hydrological Input. doi:10.2139/ssrn.4454256 SSRN preprint.
- [31] Kiran-Kumar Muniswamy-Reddy, David A. Holland, Uri Braun, and Margo Seltzer. 2006. Provenance-aware storage systems. In *Proceedings of the Annual Conference on USENIX '06 Annual Technical Conference* (Boston, MA) (ATEC '06). USENIX Association, USA, 4.
- [32] Thomas F. J.-M. Pasquier, Jatinder Singh, David Eyers, and Jean Bacon. 2017. Camflow: Managed Data-Sharing for Cloud Services. *IEEE Transactions on Cloud Computing* 5, 3 (2017), 472–484. doi:10.1109/TCC.2015.2489211
- [33] Quan Pham, Tanu Malik, and Ian Foster. 2014. Auditing and Maintaining Provenance in Software Packages. In *Revised Selected Papers of the 5th International Provenance and Annotation Workshop on Provenance and Annotation of Data and Processes - Volume 8628* (Cologne, Germany) (IPAW 2014). Springer-Verlag, Berlin, Heidelberg, 97–109. doi:10.1007/978-3-319-16462-5_8
- [34] Lukas Rupperecht, James C. Davis, Constantine Arnold, Alexander Lubbock, Darren Tyson, and Deepavali Bhagwat. 2019. Ursprung: Provenance for Large-Scale Analytics Environments. In *Proceedings of the 2019 International Conference on Management of Data* (Amsterdam, Netherlands) (SIGMOD '19). Association for Computing Machinery, New York, NY, USA, 1989–1992. doi:10.1145/3299869.3320235
- [35] Mahadev Satyanarayanan, Jan Harkes, and James Blakley. 2023. Towards Reproducible Execution of Closed-Source Applications from Internet Archives. In *Proceedings of the 2023 ACM Conference on Reproducibility and Replicability* (Santa Cruz, CA, USA) (ACM REP '23). Association for Computing Machinery, New York, NY, USA, 15–26. doi:10.1145/3589806.3600035
- [36] Diomidis Spinellis, Panos Louridas, and Maria Kechagia. 2016. The evolution of C programming practices: a study of the Unix operating system 1973–2015. In *Proceedings of the 38th International Conference on Software Engineering* (Austin, Texas) (ICSE '16). Association for Computing Machinery, New York, NY, USA, 748–759. doi:10.1145/2884781.2884799
- [37] Ji Sun, Jintao Zhang, Zhaoyan Sun, Guoliang Li, and Nan Tang. 2021. Learned cardinality estimation: a design space exploration and a comparative evaluation. *Proc. VLDB Endow.* 15, 1 (Sept. 2021), 85–97. doi:10.14778/3485450.3485459
- [38] Miklós Szeredi. 2005. FUSE: Filesystem in Userspace. <https://github.com/libfuse/libfuse>. Accessed 2024.
- [39] Bharath Kumar Reddy Vangoor, Vasily Tarasov, and Erez Zadok. 2017. To FUSE or Not to FUSE: Performance of User-Space File Systems. In *15th USENIX Conference on File and Storage Technologies (FAST 17)*. USENIX Association, Santa Clara, CA, 59–72. <https://www.usenix.org/conference/fast17/technical-sessions/presentation/vangoor>
- [40] Anthony Westbrook. 2021. *RepeatFS: A File System Providing Reproducibility Through Provenance and Automation*. Doctoral dissertation. University of New Hampshire. <https://scholars.unh.edu/dissertation/2640>
- [41] Anthony Westbrook, Jonathan Ramsdell, Todd Schuelke, Leslie Normington, Ryan D. Bergeron, Wilfred K. Thomas, and Matthew D. MacManes. 2017. PALADIN: protein alignment for functional profiling whole metagenome shotgun data. *Bioinformatics* 33, 10 (2017), 1473–1478. doi:10.1093/bioinformatics/btx021
- [42] Huan Xie, Yan Lei, Meng Yan, Yue Yu, Xin Xia, and Xiaoguang Mao. 2022. A universal data augmentation approach for fault localization. In *Proceedings of the 44th International Conference on Software Engineering* (Pittsburgh, Pennsylvania) (ICSE '22). Association for Computing Machinery, New York, NY, USA, 48–60. doi:10.1145/3510003.3510136
- [43] Jun Zhao, Carole Goble, Robert Stevens, and Daniele Turi. 2008. Mining Taverna's semantic web of provenance. *Concurr. Comput.: Pract. Exper.* 20, 5 (April 2008), 463–472.
- [44] Alexander Zhou, Yue Wang, and Lei Chen. 2021. Butterfly counting on uncertain bipartite graphs. *Proc. VLDB Endow.* 15, 2 (Oct. 2021), 211–223. doi:10.14778/3489496.3489502